ChatGPT

SECTION 0 — SCOPE & ASSUMPTIONS

0.1 Clarify the scope: Aureon's reconciliation engine will target institutional asset managers and related entities, including Portfolio Management Services (PMS), AIFs (alternative investment funds), hedge funds, mutual funds, custodians, wealth managers, and family offices. We will focus on common liquid asset classes: listed equities, ETFs, mutual funds, fixed-income (bonds), FX/foreign exchange transactions, simple derivatives (exchange-traded futures/options), and cash. The design should account for the major markets globally (US, UK/EU, India, APAC), including typical settlement cycles (T+1 in the US for equities [1]; T+2 in other markets by default), relevant regulations (e.g. SEC/FINRA, UCITS/EMIR, SEBI, MAS/SFC), and variations such as India's NSDL/CDSL equities. We assume Aureon will compare internal (OMS/IBOR) data against external sources (broker/contract notes, custodian statements, bank ledger, fund admin reports, etc.) as part of a daily (batch) reconciliation process, with optional intraday updates when files arrive.

0.2 Explicit assumptions:

- **Data availability:** We assume the ingestion layer produces normalized tables with key fields. For trades: fields like `trade_id`, `trade_date`, `settlement_date`, `symbol` / `isin` (or ticker), `side` (buy/sell), `quantity`, `price`, `gross_amount`, `net_amount`, `commission` / `fees_total`, `trade_currency`, `settlement_currency`, `broker_code`, and tenant_id. For cash transactions: `cash_id`, `value_date`, `booking_date`, `amount`, `currency`, `description` / `reference`, `status`, etc. For positions: `as_of_date`, `symbol` / `isin`, `quantity`, `market_price`, `total_value`, `currency`, etc. For NAV: `nav_date`, `fund_id`, `nav_value`, `aum`, `currency`. For corporate actions: `ex_date`, `record_date`, `symbol`, `isin`, `action_type`, `ratio` or `cash_amount`, etc. We assume data is reasonably clean (fields coerced to proper types, missing defaults filled), but Tier-0 checks will flag obvious quality issues (invalid dates, missing ISIN, etc.).
- **Frequency:** Reconciliations run daily (after market close) per fund/account. Intraday feeds may trigger incremental rules (e.g. new trade or cash file). The engine should support both batch (reconcile all from last run date) and transactional modes (reconcile on each upload).
- **Settlement cycles:** US equities are now T+1 (one business day settlement [1]); most other major markets (EU, India, UK, HK) are T+2. Fixed income and some Asia markets (e.g. Japan equities) have their own cycles (to be noted per fund), but we will assume T+2 as default unless overridden. Our rules will allow configurable date offsets (e.g. a 1–2 day window for matching trade vs cash or position dates). Corporate action record/payment lags (ex-date vs pay-date) will also be accounted for by specific rules (see Section 3).
- **Corporate actions coverage:** We will handle standard cases: stock splits (incl. reverse splits), bonus/scrip issues, cash dividends, rights issues, and simple mergers/ spin-offs at a high level. For example, after a 2-for-1 split on an ex-date, quantity doubles and price halves in the holdings. We will **not** delve into extremely rare or complex corporate actions (e.g. spinoffs into multiple tickers, complicated reorgs) beyond basic ratio adjustments. Record/ex-dates and pay-dates for corporate events will be treated in rules to avoid false breaks (see Section 3).

Geography-specific notes: If a rule or parameter differs by region, it will be explicitly labeled. For instance, Indian markets impose a Securities Transaction Tax (STT) on equity trades (buy side and/sell side) and GST on brokerage; UK/EU may have stamp taxes; US has SEC fees on stock sells, etc. Our design will allow per-tenant/regional configuration for such charges.

SECTION 1 — RECONCILIATION DOMAINS & FLOWS

We break down the overall reconciliation universe into core domains that Aureon should support, each comparing a pair (or set) of sources. In general, **reconciliation** means comparing and matching records from *internal books* (trades, cash ledgers, position book, NAV calculations) with *external records* (custodian statements, broker confirmations, bank statements, fund administrator reports) to identify mismatches. Good matches (no breaks) mean the numbers align within allowable tolerances; a "break" is any unmatched or out-of-tolerance discrepancy requiring investigation. The main domains are:

- **Trade ↔ Cash:** Reconcile executed trade transactions against cash movements. This ensures that for every buy there was the corresponding cash debit, and for every sell a cash credit, with correct amounts, currencies and dates. Typical sources: broker trade blotters or contract notes (CSV/XLS/PDF) and bank/custodian cash statements (CSV, MT940 format, etc.). Good: each trade's net cash (quantity×price ± fees) is reflected by one or more cash entries on settlement date. A break: e.g. cash entry missing or duplicate, wrong amount (possibly due to fees), or date off (trade on T, cash on T+2). Common triggers: unsettled trades, fees/taxes missing, FX conversion issues.
- **Trade ↔ Custodian/Contract Notes:** Match internal trades (or OMS orders) to what the broker or custodian reports. For instance, a broker's contract note (often a PDF or text file) is compared against the trade blotter. Good: quantity, price, symbol/ISIN and amounts agree. A break: mismatched trade details (price or qty discrepancy), unacknowledged trades, or duplicates. File sources include broker contract notes, OMS export vs custodian's trade-tracking files. (This helps catch data-entry errors and is sometimes called "trade confirmation reconciliation.")
- **Positions ↔ Custodian (Holdings):** Compare internal positions/holdings (IBOR) to the custodian's holdings or DP (depository participant) statements. The key fields: instrument identifier, quantity and market value. Good: totals match (allowing for expected differences from unsettled trades or pending actions) [2] [3]. A break: quantity mismatch (we think we hold 100 shares, custodian shows 102), likely due to an un-recorded split, corporate action, or un-settled trade. Typical inputs: custodian's daily holdings report or statement (CSV/Excel), DP statement (for Indian demat), vs internal end-of-day book.
- **NAV & AUM checks:** Validate internally computed NAV/AUM against the fund administrator's or custodian's NAV report [4]. Good: NAV (per unit) and total AUM match within rounding tolerances. A break means the NAV differs materially (maybe due to missing cash, mispriced asset, or fee not applied). Sources: fund admin NAV report (CSV/Excel), internal NAV calculation from holdings+cash. This is crucial for investor reporting/regulatory compliance [4].
- **Fees & Charges:** Check that all expected fees and taxes have been applied and recorded. Examples: management fees (e.g. 1.5% of AUM monthly), brokerage commissions (from broker invoices vs trade records), exchange/clearing charges (SEC fees, transaction levies, stamp duty, GST, etc.), custodian custody charges, etc. Good: the amounts in ledger match computed or invoiced amounts. A break: missing or extra fee line (e.g. custodian levies missed, or broker charged unexpected GST). Data sources include fee schedules, broker invoice files, custodian fees CSV, and internal fee logs.
- **Corporate Actions Impact:** Verify that dividends, splits, bonuses, rights, mergers, etc. are correctly reflected in holdings and cash. E.g., after a 2-for-1 stock split, the quantity should double and average cost halve. Good: positions updated and any cash (e.g. dividends) received as expected [5]. Breaks: an unprocessed bonus left shares short, or a dividend paid but not in cash records. Corp action announcements (fix files or API data) are matched to effects in the portfolio.

- **FX & Multi-currency Valuation:** In multi-currency portfolios, reconcile that foreign-currency trades and cash are translated correctly into the base currency. E.g., a trade in EUR should have a USD equivalent using the agreed FX rate. Good: internal FX conversion (often EOD rate) matches custodian's. A break: using a wrong FX or stale price, leading to P&L mismatch. Sources: custodian currency reports, FX rate feeds, internal conversion logic.
- **Suspense & Break Management:** Not a reconciliation per se but a category: any cash or positions that fail to match go into a suspense/unmatched bucket for manual review. Good: the system flags these and they eventually get resolved. A "break" in this context means an item is unresolved (e.g. cash sitting unallocated, or a security in holding with no origin). Typically sources are the output of all above matches; the suspense account or break logs are the exceptions.

In summary, *good reconciliation* means no significant unexplained differences across these domains (currency balances align, share quantities match, fees are accounted for, etc.). A *break* means a discrepancy that could affect P&L, NAV accuracy, or regulatory reporting. We will design each domain's rules to detect typical break patterns (e.g., missing cash vs missing trade, wrong rates, unposted charges).

SECTION 2 — DETAILED RULE CATALOG (THE HEART)

Below are representative reconciliation rules for each domain. Each rule is classified by type (deterministic exact match, tolerance-based check, data quality, or edge-case), and includes logic pseudocode, tolerances, severity, and examples. These reflect industry practice (custodian/fund admin reconciliations, OMS/custodian logic) and should be implementable in SQL or procedural code.

2.1 Trade ↔ Cash Reconciliation Rules

- id: **TRADE_CASH_001**
  domain: Trade-Cash
  name: Exact Gross Amount Match
  type: DETERMINISTIC
  description: Ensure each trade's gross amount matches an equal and opposite cash entry. For a BUY trade, the bank cash movement (credit or debit) should equal quantity×price (gross) outflow, and similarly for a SELL.
  inputs:
    - trade.gross_amount: total trade value (incl. without fees)
    - trade.side: BUY/SELL
    - cash.amount: cash transaction amount (debit as negative, credit as positive)
    - cash.currency: currency of cash movement
    - trade.trade_currency: trade currency
    - cash.value_date: date cash moved
    - trade.settlement_date: expected settlement date
      logic_pseudocode_sql: | -- match trade to cash by exact amount IF trade.side = 'BUY' THEN expected_amount = -ABS(trade.gross_amount) ELSE expected_amount = ABS(trade.gross_amount) END IF MATCH if cash.amount = expected_amount AND cash.currency = trade.trade_currency AND cash.value_date = trade.settlement_date
      suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0

deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: This is a strict equality check; fully SQL-compatible.
severity: level: CRITICAL
impact_if_broken: P&L error or failed settlement if trade is not matched to cash, regulatory (NAV) inaccuracy.
example_breaks:

- "Example: A BUY trade of 100 shares @ $10 yields gross_amount $1000. No cash of $1000 debit appears on T+1; instead, only $950 is recorded. This $50 gap (possibly an unrecorded commission) is a break."
- "Example: A SELL trade expected $500 credit appears as $500 debit (sign error) due to system inversion; trade and cash don't cancel out, causing an imbalance."
  example_fix:
- "Ops would split out the missing $50 commission: adjust internal trade to include $50 fee, matching $950 cash, then record $50 commission entry separately to balance."
- "For the sign mismatch, correct the cash entry's debit/credit flag so the $500 appears as a credit, clearing the break."

- id: **TRADE_CASH_002**
  domain: Trade-Cash
  name: Net Amount (TradeNet = CashNet)
  type: DETERMINISTIC
  description: Check that the trade's net amount (including fees/commissions as recorded in trade blotter) equals the cash movement. This catches cases where fees are integrated.
  inputs:

  - trade.net_amount: net trade value (gross ± fees)
  - trade.side
  - cash.amount
  - cash.currency
  - trade.trade_currency
  - trade.settlement_date, cash.value_date
    logic_pseudocode_sql: | IF trade.side = 'BUY' THEN expected_cash = -ABS(trade.net_amount) ELSE expected_cash = ABS(trade.net_amount) END IF MATCH if cash.amount = expected_cash AND cash.currency = trade.trade_currency AND cash.value_date = trade.settlement_date suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: This is also strict; often identical to gross match if net= gross. Use if fees are included in net_amount.
    severity: level: CRITICAL
    impact_if_broken: Same as above (P&L/NAV impact).
    example_breaks:
  - "Example: A $1000 buy trade has $10 fee; trade.net_amount = $1010. If cash only shows $1000 debit, the $10 fee is missing, causing a break."
  - "Example: A $500 sell with $5 rebate has trade.net_amount $495 credit; cash is $500, $5 too high, indicating a fee misunderstanding."
    example_fix:
  - "Add the missing $10 commission entry to the cash ledger (debit $1000 + debit $10 fee) to reconcile to trade."
  - "Adjust internal trade record to properly include the $5 rebate so that net matches observed cash."

- id: **TRADE_CASH_003**
  domain: Trade-Cash
  name: Currency Match
  type: DETERMINISTIC
  description: Verify that the trade currency matches the cash currency. A trade executed in USD should settle with USD cash.
  inputs:
    - trade.trade_currency
    - cash.currency
      logic_pseudocode_sql: | MATCH if cash.currency = trade.trade_currency
      suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Straightforward code check. If currencies differ, it's a break or requires FX logic.
      severity: level: HIGH
      impact_if_broken: Could indicate data error or misinterpreted FX trade.
      example_breaks:
    - "Example: Trade recorded in EUR, but matching cash entry is in USD; likely an FX mismatch or data error."
    - "Example: A USD-denominated sell trade was erroneously settled in GBP cash, likely due to wrong account selection."
      example_fix:
    - "Ops identifies the currency mismatch, then uses FX translation to link the trade and cash (see FX rules) or corrects the trade currency data."
    - "In multi-currency funds, such mismatches might be flagged for manual review; one solution is to convert one side to the other via the known FX rate."

- id: **TRADE_CASH_004**
  domain: Trade-Cash
  name: Settlement Date Alignment
  type: TOLERANCE
  description: Check that the cash transaction date is within an acceptable offset of the trade's settlement date. Typically, cash for a trade should occur on T+1 or T+2 depending on asset/market. Allow small offsets for weekends/holidays.
  inputs:
    - trade.settlement_date
    - cash.value_date
      logic_pseudocode_sql: | days_diff = DATEDIFF(day, trade.settlement_date, cash.value_date) MATCH if ABS(days_diff) <= allowed_offset
      suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 1 # e.g., 1-day tolerance deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: A purely date-offset check; holidays/weekends cause 1-2 day differences.
      severity: level: MEDIUM
      impact_if_broken: Usually operational (settlement timing) rather than P&L, but still flagged.
      example_breaks:
    - "Example: Trade settlement_date is 2025-12-01, but cash value_date is 2025-12-05 (4 days later). Possibly a delayed settlement or data issue."

- "Example: A T+1 market had cash on T+2, indicating a holiday gap; if unaccounted, triggers a break."
  example_fix:
- "Check holiday calendar: adjust tolerance (REGULATION DETAIL — TO BE VERIFIED). If an extra day, tag as weekend effect. Otherwise, investigate settlement failure."
- "If consistently 1-day off, consider automating allowed_offset=2 for that market to auto-resolve similar cases."

- id: **TRADE_CASH_005**
  domain: Trade-Cash
  name: Quantity-Sign Check
  type: DATA_QUALITY
  description: Ensure that trade side (BUY vs SELL) correlates to the sign of the cash amount. E.g., buys should result in cash outflows (negative or debit), sells in inflows. This is more of a sanity check.
  inputs:

  - trade.side
  - trade.quantity
  - cash.amount
    logic_pseudocode_sql: | IF trade.side = 'BUY' AND cash.amount >= 0 THEN BREAK IF trade.side = 'SELL' AND cash.amount <= 0 THEN BREAK suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Simple sign check. Negative vs positive conventions vary by system.
    severity: level: HIGH
    impact_if_broken: Indicates incorrect data entry (e.g., cash flagged wrong). May cause P&L sign flips.
    example_breaks:
  - "Example: A BUY of 100 shares should create a -$1000 cash entry, but cash.amount is +1000 (likely mis-flagged as a credit instead of a debit)."
  - "Example: A SELL trade should credit cash, but the system recorded it as a debit, implying cost instead of proceeds."
    example_fix:
  - "Ops flips the sign on the cash entry (debit vs credit) to match the trade side, reconciling the break."
  - "If systemic, adjust import mapping so future buy trades generate negative cash, sells positive."

- id: **TRADE_CASH_006**
  domain: Trade-Cash
  name: Commission and Fees Presence
  type: DATA_QUALITY
  description: For trades, verify that if the trade record lists explicit commission/fees, the cash postings also include the same. And vice versa: unexpected cash fees should appear in trade.
  inputs:

  - trade.commission (or fees_breakup fields)
  - cash.description or fees fields
  - trade.net_amount, trade.gross_amount
    logic_pseudocode_sql: | IF trade.commission > 0 THEN ASSERT sum(cash.amount where description CONTAINS 'commission' OR 'fee') = trade.commission END IF

suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Could be a tolerance rule if minor rounding is allowed (see TOLERANCE rules).
severity: level: MEDIUM
impact_if_broken: Fees not tracked means cost basis or P&L is incorrect.
example_breaks:

- "Example: Trade indicates $50 commission, but no $50 expense appears in cash transactions. Instead, the cash is off by $50."
- "Example: A cash debit of $20 marked 'stamp duty' appears, but trade.commission is $0; suggests an unstated fee was taken."

example_fix:

- "Split the unmatched $50 from cash into a separate commission ledger entry; update trade or fee schedule so the commission is logged internally."
- "Add a virtual trade fee line for the $20 stamp duty, allocating it properly so NAV calculation includes that charge."

- id: **TRADE_CASH_007**
  domain: Trade-Cash
  name: Partial Settlement Handling
  type: EDGE_CASE
  description: Handle trades settled in parts. A single trade may result in multiple cash entries (e.g., partial fills or multiple payments). The sum of partial cash amounts must equal the full trade amount.
  inputs:

  - trade.gross_amount
  - set of cash.amount entries linked (or eligible by date/currency)
  - trade.settlement_date, cash.value_date
    logic_pseudocode_sql: | Find cash records C1..Cn with dates = trade.settlement_date and currency = trade_currency. IF SUM(C1..Cn.amount) = (trade.side = BUY ? -ABS(trade.gross_amount) : ABS(trade.gross_amount)) THEN match.
    suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
    deterministic_or_ai: should_live_in: "HYBRID" notes: The rule engine can sum matching cash entries, but linking multiple requires either a group key or using AI to suggest matches if ambiguous.
    severity: level: HIGH
    impact_if_broken: Partial fill mismatches can leave orphaned cash or trades.
    example_breaks:
  - "Example: A buy trade for $1000 was settled in two installments: $600 and $400. The rule must match both to clear the trade. If only one $600 is matched, a $400 gap remains."
  - "Example: A trade reversed partially ($100 out of $500) leaves $100 in cash still unmatched if both payments aren’t captured."
    example_fix:
  - "Ops manually allocates both $600 and $400 cash entries to the trade. The system should auto-group them next time."
  - "If no clear link, create a suspense break for the unmatched $400 and investigate separately."

- id: **TRADE_CASH_008**
  domain: Trade-Cash
  name: Multiple Trades to One Cash Entry (Netting)
  type: EDGE_CASE

description: In net settlement, multiple trades (often buys and sells on same day) may net to a single cash transfer. For example, a trader's netted trades may result in one combined cash. Detect if the sum of trade nets equals the cash.
inputs:

- multiple trade.gross_amount/net_amount for same account/date
- single cash.amount
  logic_pseudocode_sql: | Identify trade group T1..Tm on same date/account. IF SUM(net_amounts of sells) – SUM(net_amounts of buys) = cash.amount (for net settlement) THEN match group.
  suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
  deterministic_or_ai: should_live_in: "HYBRID" notes: Rule engine can try small combinations, but AI may help identify which trades net to which cash if not one-to-one.
  severity: level: HIGH
  impact_if_broken: Breaks in netting cause false unmatched items; critical for correct cash forecasting.
  example_breaks:
- "Example: Three sell trades ($100, $200, $300) and two buys ($150, $50) occur. Net cash = $400. If only a $400 deposit is seen, the system should net them; if not, it flags all five trades as missing."
- "Example: End-of-day net settle has $0 cash but multiple offsetting trades; if not recognized, each trade seems unmatched."
  example_fix:
- "Group the five trades as described and match to the single $400 cash. Our engine would then no longer break them individually."
- "Manual allocation: Ops nets the trades, or escalates to treasury for validation of net settlement."

- id: **TRADE_CASH_009**
  domain: Trade-Cash
  name: FX Conversion Consistency
  type: TOLERANCE
  description: For cross-currency trades, allow matching if cash is in a different currency via an FX conversion. E.g., a USD trade settled with EUR cash. The engine should convert one side using the official rate and compare.
  inputs:

  - trade.gross_amount, trade.trade_currency
  - cash.amount, cash.currency
  - FX rates (internal or custodian-provided) for trade_date or settlement_date
    logic_pseudocode_sql: | IF trade.currency != cash.currency THEN converted_cash = cash.amount * FX_RATE(cash.currency -> trade.currency on trade_date) MATCH if converted_cash = (trade.side = BUY ? -trade.gross_amount : trade.gross_amount) within tolerance
    suggested_tolerances: amount_abs: 0.01
    amount_pct: 0.1%
    date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: FX rates must be sourced (see Section 4). This is still deterministic once rate is known.
    severity: level: MEDIUM
    impact_if_broken: Without FX logic, cross-currency trades appear broken. PnL could be off

by currency rounding.
example_breaks:
- "Example: A $1000 USD sale settles with €910. If the USD/EUR rate used was 1.10, €910 = $1001; a $1 difference arises due to rounding."
- "Example: A EUR trade settles in USD but using stale (T-1) rate, causing a cent difference."

example_fix:
- "Apply correct FX (e.g. mid-rate) to convert €910 to $1001 and allow a $1 tolerance. Flag rounding errors but mark matched."
- "If rate source was wrong, update FX feed or pick custodian’s reference rate to match exactly."

- id: **TRADE_CASH_010**
  domain: Trade-Cash
  name: Duplicate Trade Entry Detection
  type: DATA_QUALITY
  description: Detect if the same trade appears more than once in the trade blotter (potentially causing one of them to have no cash match). E.g., an accidental double entry.
  inputs:
    - trade.trade_date, symbol, quantity, price, broker_code
      logic_pseudocode_sql: | GROUP BY trade_date,symbol,quantity,price,broker_code HAVING COUNT(trade_id) > 1
      suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: This is a quality check to catch duplicates.
      severity: level: MEDIUM
      impact_if_broken: Duplicate trades could mask true cash matching (one copy unmatched).
      example_breaks:
    - "Example: Trade blotter shows TradeID 1001 and 1002 both for 100 shares @ $10 on same date. Only one corresponding cash of $1000 exists. One trade will be marked a break."
      example_fix:
    - "Identify and remove or merge the duplicate trade. Mark one as correction of the other, then match cash normally."

- id: **TRADE_CASH_011**
  domain: Trade-Cash
  name: Trade Cancellation/Reversal
  type: EDGE_CASE
  description: Handle trades that were cancelled or reversed. If a trade is reversed, a counter-trade or adjustment should cancel its cash. The reconciliation should recognize and net these.
  inputs:
    - trade.status or a flag for cancellation
    - if reversal trades exist (negative quantity)
      logic_pseudocode_sql: | IF trade.status = 'CANCELLED' OR trade.quantity < 0 THEN EXPECT matching inverse cash or no net effect. END IF
      suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Cancels often result in offsetting trades/cash on same day.
      severity: level: MEDIUM

impact_if_broken: Without recognizing cancellations, the system would report false breaks (unsettled trades).
example_breaks:

- "Example: A $500 buy trade is cancelled later. A $500 credit should appear in cash (return of money). If not matched as a reversal, it looks like $500 unexplained cash."
- "Example: A short sale is covered the same day; if not netted, both trades seem individually unmatched."

example_fix:

- "Rule-engine pairs cancelled trades with their reversal cash entries so neither counts as a real position. The cash refund is treated as settling the original trade."

- id: **TRADE_CASH_012**
  domain: Trade-Cash
  name: Unmatched Cash (Trade Missing)
  type: TOLERANCE
  description: Detect cash entries that have no corresponding trade. Small cash items (e.g., $1 remainder) may be tolerable (rounding), others may indicate missing trade data.
  inputs:

  - cash records not linked to any trade after matching passes
    logic_pseudocode_sql: | FOR each cash.amount IF no trade found and ABS(cash.amount) > min_tolerance THEN FLAG break suggested_tolerances: amount_abs: 1.0 amount_pct: 0.1% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Use small tolerance for micro-remainders (e.g. $1) possibly ignore. Larger: need human review.
    severity: level: LOW
    impact_if_broken: Likely an operational detail; too many could skew cash reporting.
    example_breaks:
  - "Example: $0.50 cash entry with no trade – likely rounding, can be ignored (within tolerance). But $50 with no trade is a break."
    example_fix:
  - "Manual: Confirm $50 was an interest credit or fee reversal and allocate it. If no reason, keep it in suspense."

- id: **TRADE_CASH_013**
  domain: Trade-Cash
  name: Currency Code Validity
  type: DATA_QUALITY
  description: Check that currency codes in trade and cash records are valid ISO codes (USD, EUR, INR, etc.) and mapped to base currency. Invalid or missing currency should be flagged.
  inputs:

  - trade.trade_currency
  - cash.currency
    logic_pseudocode_sql: | IF trade.trade_currency NOT IN valid_currencies OR cash.currency NOT IN valid_currencies THEN BREAK
    suggested_tolerances: amount_abs: 0 amount_pct: 0% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: A data cleanliness rule to catch bad currency entries.
    severity: level: MEDIUM
    impact_if_broken: Mis-coded currencies can cause false mismatches and FX errors.
    example_breaks:

- "Example: Trade.currency is blank or “USX” (typo). The cash is USD. The rule flags the bad code and mismatched currency."
  example_fix:
- "Correct currency code (e.g. “USD”), then allow the trade/cash to match normally."

2.2 Trade ↔ Custodian / Contract Note Rules

- id: **TRADE_CUST_001**
  domain: Trade-Custodian
  name: Trade Detail Exact Match
  type: DETERMINISTIC
  description: Ensure key trade fields (ISIN, quantity, price, side, settlement_date) match between the internal trade blotter and the custodian or broker’s record.
  inputs:
  - trade.trade_id / external_ref
  - trade.symbol/ISIN, trade.quantity, trade.price, trade.side, trade.settlement_date
  - custodian.symbol/ISIN, custodian.quantity, custodian.price, custodian.side, custodian.settlement_date
    logic_pseudocode_sql: | JOIN trades t ON custodian_ref CHECK t.symbol = c.symbol AND t.quantity = c.quantity AND t.price = c.price AND t.side = c.side suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Straightforward one-to-one match; typically both systems use trade references.
    severity: level: CRITICAL
    impact_if_broken: Unmatched trade confirmations can lead to unbooked trades or False P&L.
    example_breaks:
  - "Example: Trade shows 100 shares @ $10, but custodian report shows 100 @ $10.01. Price discrepancy may be due to exchange fees not captured."
  - "Example: Trade is BUY, but contract note side says SELL (data entry error), causing a straight mismatch."
    example_fix:
  - "Human checks which is correct (broker invoice vs OMS). Correct the trade or inform broker. If due to fees, split them out."

- id: **TRADE_CUST_002**
  domain: Trade-Custodian
  name: Identifier Consistency (ISIN/Ticker)
  type: DATA_QUALITY
  description: Ensure instrument identifiers align. For example, if internal uses ticker and custodian uses ISIN, these should map. Check for common aliases or known symbol conversions.
  inputs:
  - trade.isin or symbol
  - custodian.isin or symbol
    logic_pseudocode_sql: | IF NOT matches(trade.ISIN, custodian.ISIN) AND NOT alias_match(trade.symbol, custodian.symbol) THEN BREAK
    suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
    deterministic_or_ai: should_live_in: "HYBRID" notes: Basic mapping might be deterministic if a symbol mapping table exists. LLM/AI may assist with fuzzy name

matching (e.g., “GOOG” vs “Alphabet”).
severity: level: HIGH
impact_if_broken: Wrong instrument match causes quantity mismatch and valuation error.
example_breaks:

- "Example: Internal has symbol ‘BRK-A’, custodian shows ‘BRK-B’ (even if quantities same). Identifier mismatch triggers a break."
- "Example: Trade records ‘TATASTEEL’ while custodian uses ISIN; if not mapped, they won’t match."

example_fix:

- "Add alias mapping (e.g., maintain an ISIN lookup). After mapping, the trade and custodian entries align."

- id: **TRADE_CUST_003**
  domain: Trade-Custodian
  name: Matching by Quantity and Net Amount
  type: TOLERANCE
  description: If trade references differ, match by key fields. For example, match on trade_date, quantity, security, and net amount. Allow small tolerances on quantity sign or rounding.
  inputs:

  - trade.trade_date, quantity, net_amount
  - custodian.trade_date, custodian.quantity, custodian.net_amount
    logic_pseudocode_sql: | MATCH if trade.trade_date = cust.trade_date AND trade.quantity = cust.quantity AND ABS(trade.net_amount - cust.net_amount) <= tol_amount
    suggested_tolerances: amount_abs: 0.01
    amount_pct: 0.01%
    date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Basic tolerance on amounts (e.g. rounding).
    severity: level: HIGH
    impact_if_broken: Prevents matching when trade IDs aren’t aligned. Useful fallback rule.
    example_breaks:
  - "Example: Trade net $1000 vs custodian $1000.01 (1 cent diff due to FX). If tol 0.01, it matches; without it, break."
  - "Example: Quantity mismatched by 1 share (e.g., trade 100 vs cust 99) - likely a true break, but with tol 0 it flags properly."

  example_fix:

  - "Increase tolerance slightly for cent differences if seen consistently. Otherwise, find out why quantity mismatched (maybe split, see corp rules)."

- id: **TRADE_CUST_004**
  domain: Trade-Custodian
  name: Contract Note Fee Check
  type: DATA_QUALITY
  description: Ensure that any broker fees reported on the contract note are recorded in the trade record. For example, if broker’s statement shows $30 commission, the trade should reflect that.
  inputs:

  - broker_fees on contract note
  - trade.commission/fees fields
    logic_pseudocode_sql: | IF broker_fees > 0 AND trade.commission != broker_fees THEN

BREAK
suggested_tolerances: amount_abs: 0 amount_pct: 0% date_offset_days: 0
deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Cross-check fee fields.
severity: level: MEDIUM
impact_if_broken: Broker could charge extra fees not captured, affecting P&L and costs.
example_breaks:

- "Example: Contract note shows $20 brokerage, but trade.commission is null. The $20 cost won't be in portfolio calculation."
  example_fix:
- "Ops enters the $20 fee into the trade record or a separate fees table. Automation could propagate known brokerage fees per broker."

- id: **TRADE_CUST_005**
  domain: Trade-Custodian
  name: Trade Date vs Execution Date
  type: DATA_QUALITY
  description: Verify the trade date (execution date) matches between internal and external sources. Some systems use trade_date vs set_date differently.
  inputs:

  - trade.trade_date
  - custodian.trade_date
    logic_pseudocode_sql: | MATCH if trade.trade_date = cust.trade_date
    suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 1 # allow next-day timestamps
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Accounts for time zone or end-of-day lags (tolerance=1 day).
    severity: level: LOW
    impact_if_broken: Usually date misalignment; often not material if within 1 day.
    example_breaks:
  - "Example: Internal system shows trade 2025-12-01, custodian file shows 2025-12-02 (reporting lag). With a 1-day offset allowed, it passes."
  - "Example: A trade report has wrong date (typo) 2025-11-30 vs actual 12-01; rule flags it if offset=0."
    example_fix:
  - "If systematic one-day off, configure offset. Otherwise, correct data source."

2.3 Positions ↔ Custodian (Holdings) Reconciliation Rules

- id: **POS_CUST_001**
  domain: Positions-Custodian
  name: Quantity Match
  type: DETERMINISTIC
  description: The total quantity of each security in the internal position should equal the quantity reported by the custodian for the same date and account.
  inputs:

  - position.as_of_date, symbol, quantity
  - custodian.as_of_date, symbol, quantity
    logic_pseudocode_sql: | MATCH if position.date = custodian.date AND position.symbol = custodian.symbol AND position.quantity = custodian.quantity
    suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0

deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Exact match. Unsettled trades may cause known differences (handled by a separate rule).
severity: level: CRITICAL
impact_if_broken: A custodian showing a different share count is a major error (unexplained asset).
example_breaks:

- "Example: Internal holds 100 shares of XYZ, custodian report shows 102. Possibly 2 shares of dividend reinvestment or unbooked bonus."

example_fix:

- "Identify cause (e.g. call bonus issue). Update internal or custodian record after investigation."

- id: **POS_CUST_002**
  domain: Positions-Custodian
  name: Market Value Tolerance
  type: TOLERANCE
  description: Compare total market value of each holding (quantity×price). Allow small differences for price source or rounding.
  inputs:

  - position.quantity, position.market_price, position.currency
  - custodian.quantity, custodian.price, custodian.currency
    logic_pseudocode_sql: | IF position.currency = custodian.currency THEN val1 = position.quantity * position.market_price val2 = custodian.quantity * custodian.price MATCH if ABS(val1 - val2) <= tolerance_amount suggested_tolerances: amount_abs: 1.0 # e.g. $1 tolerance amount_pct: 0.1%
    date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Different price sources (IBOR vs custodian) often differ by small amounts or rounding.
    severity: level: MEDIUM
    impact_if_broken: Large valuation gaps could hide pricing errors or corporate actions not applied.
    example_breaks:
  - "Example: 100 shares @ $10 vs custodian 100 @ $10.01. Value diff $1 (1%), if tol 1, okay; if tol 0, flagged."
  - "Example: Multi-currency position: internal in USD, custodian in INR (use FX adjustment rule). If currency mismatched, skip this rule."
    example_fix:
  - "Adjust tolerance if consistently a cent or two. Otherwise re-run price fetch or check market data timing."

- id: **POS_CUST_003**
  domain: Positions-Custodian
  name: Unsettled Trades Effect
  type: DATA_QUALITY
  description: Recognize that unsettled trades will cause position differences. If the difference between internal and custodian equals recently traded quantity, this is expected (not a real break).
  inputs:

  - trade book (recent trades), internal position, custodian position
    logic_pseudocode_sql: | unsettled_qty = sum(trade.quantity for trade.settlement_date >

as_of_date) MATCH if (internal.quantity + unsettled_qty) = custodian.quantity
suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Must use trade book to adjust.
severity: level: LOW
impact_if_broken: Not a true break; indicates pending settlement. Helps avoid false positives.
example_breaks:

- "Example: Internal shows 0 shares (no settled trades yet), custodian shows 100 on T+2 (trade on T). If recognized as unsettled, no break."
  example_fix:
- "Design rule to automatically clear this case rather than investigate."

- id: **POS_CUST_004**
  domain: Positions-Custodian
  name: Corporate Action Consistency
  type: EDGE_CASE
  description: Detect if a known corporate action (split, bonus, etc.) occurred and adjust expected positions accordingly. For example, after a 2:1 split, custodian positions double while internal should double.
  inputs:

  - corp_action table: symbol, ex_date, ratio, type
  - position.quantity (pre- and post-ex-date), custodian.quantity
    logic_pseudocode_sql: | IF date = corp.ex_date THEN expected_internal_qty = old_qty * split_ratio MATCH if expected_internal_qty = custodian.quantity
    suggested_tolerances: amount_abs: 0.0 amount_pct: 0.0% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Needs up-to-date corp action data (see Section 3). If unknown action, may rely on AI to guess.
    severity: level: HIGH
    impact_if_broken: Missing a split or bonus leads to wrong holdings and NAV.
    example_breaks:
  - "Example: A 10-for-1 split on ex-date: internal still 1000 shares (pre-split); custodian shows 10000 after. Rule recognizes split and verifies. If mis-aligned, break flagged."
    example_fix:
  - "If missed, manually apply split in system (update quantity and cost basis). System should then mark resolved."

- id: **POS_CUST_005**
  domain: Positions-Custodian
  name: Price Source Discrepancy
  type: TOLERANCE
  description: Sometimes internal positions use a different pricing source than custodian (e.g. Bloomberg vs custodian). Allow small percentage difference in market_price.
  inputs:

  - position.market_price
  - custodian.price
  - position.quantity (for context)
    logic_pseudocode_sql: | IF same currency THEN diff_pct = ABS(position.market_price - custodian.price)/position.market_price*100 MATCH if diff_pct <= 0.5%
    suggested_tolerances: amount_abs: 0.0 amount_pct: 0.5% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Tolerance set small (e.g. 0.5%).

severity: level: LOW
impact_if_broken: Minor pricing differences; large ones might indicate stale price.
example_breaks:
- "Example: Internal price $10.00, custodian $10.03 (0.3% diff) passes. If 1% diff, rule flags need check."
example_fix:
- "If regularly one source is known to run 0.5% high, adjust tolerance or data source."

- id: **POS_CUST_006**
  domain: Positions-Custodian
  name: Missing Price or Zero Value
  type: DATA_QUALITY
  description: If either system reports a security with no market price or value (e.g. $0), flag it.
  inputs:

    - position.market_price, custodian.price, position.total_value, custodian.total_value
      logic_pseudocode_sql: | IF position.total_value <= 0 OR custodian.total_value <= 0 THEN BREAK
      suggested_tolerances: amount_abs: 0 amount_pct: 0% date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Signifies missing data or delisted security.
      severity: level: MEDIUM
      impact_if_broken: Missing valuations lead to NAV error.
      example_breaks:
    - "Example: Position shows market_price null => total_value 0; custodian has a value. The rule flags this as incomplete."
      example_fix:
    - "Fill in price from a market data service or mark position as illiquid, then re-run reconcile."

- id: **POS_CUST_007**
  domain: Positions-Custodian
  name: Negative or Short Holding Check
  type: DATA_QUALITY
  description: Identify if internal positions have negative quantity (short) not mirrored on custodian, or vice versa. Unless for approved short strategies, this is an anomaly.
  inputs:

    - position.quantity, custodian.quantity
      logic_pseudocode_sql: | IF (position.quantity < 0 AND custodian.quantity >= 0) OR (custodian.quantity < 0 AND position.quantity >= 0) THEN BREAK
      suggested_tolerances: amount_abs: 0 amount_pct: 0% date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Simply flags opposite-sign quantities.
      severity: level: HIGH
      impact_if_broken: Could mean a missing short sale booking or reconciliation error.
      example_breaks:
    - "Example: Internal system shows -100 shares (short), but custodian doesn’t allow short (custodian shows +100), signaling a problem."
      example_fix:
    - "Validate short sale. If it’s valid, adjust custodian via borrow; if not, investigate entry error."

- id: **POS_CUST_008**
  domain: Positions-Custodian
  name: Multi-Currency Valuation Check
  type: TOLERANCE
  description: For positions held in foreign currency, ensure conversion to base currency (if needed) uses consistent rates. Check that total base-currency value aligns.
  inputs:

    - position.total_value & currency
    - custodian.total_value & currency
    - FX rate to base currency
      logic_pseudocode_sql: | IF position.currency != base_currency THEN base_val_internal = position.total_value * FX(position.currency->base) ELSE base_val_internal = position.total_value DO same for custodian MATCH if ABS(base_val_internal - base_val_custodian) <= tolerance_abs suggested_tolerances: amount_abs: 1.0 amount_pct: 0.2% date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Mixed currencies require FX normalization.
      severity: level: MEDIUM
      impact_if_broken: FX mismatches cause AUM/NAV discrepancies.
      example_breaks:
    - "Example: EUR holding valued €1000 on books vs €990 on custodian; at €1.10 rate, $1000 vs $990, $10 difference flagged (if tol $5, flags break)."
      example_fix:
    - "Update to use custodian's end-of-day FX rate to eliminate the $10 gap."

2.4 NAV & Valuation Checks

- id: **NAV_001**
  domain: NAV-AUM
  name: NAV per Unit Match
  type: TOLERANCE
  description: Verify that the internally calculated Net Asset Value (per share/unit) matches the administrator's published NAV within a tight tolerance.
  inputs:

    - nav_internal, nav_admin (each in base currency)
      logic_pseudocode_sql: | diff = ABS(nav_internal - nav_admin) MATCH if diff <= tolerance_pct * nav_admin
      suggested_tolerances: amount_abs: 0.001 amount_pct: 0.01%
      date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: NAV is critical metric; tiny percentage differences may be due to rounding or small unbooked items.
      severity: level: CRITICAL
      impact_if_broken: Direct investor reporting error/regulatory failure (NAV misstatement).
      example_breaks:
    - "Example: Admin NAV is $10.000, internal is $9.995. Difference 0.05% flagged if tol=0.01%."
    - "Example: NAVs match exactly—no break."
      example_fix:

- "Investigate missing cash or fees; likely a small accrual or rounding. Fix data so NAV aligns."

- id: **NAV_002**
  domain: NAV-AUM
  name: AUM Total Match
  type: TOLERANCE
  description: Compare total AUM (All Assets Under Management) between internal books and administrator report.
  inputs:

  - aum_internal, aum_admin
    logic_pseudocode_sql: | diff = ABS(aum_internal - aum_admin) MATCH if diff <= tolerance_pct * aum_admin
    suggested_tolerances: amount_abs: 10.0 amount_pct: 0.1%
    date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Often similar to NAV check; AUM may include different classes (cash incl. or excl.), so tolerance a bit larger.
    severity: level: HIGH
    impact_if_broken: AUM mismatch can affect fee calc and reporting.
    example_breaks:
  - "Example: AUM $100M vs $100.2M (0.2% diff) flagged if tol 0.1%."
    example_fix:
  - "Check inclusion of accrued income, or underlying fund asset counts. Adjust definitions if needed."

- id: **NAV_003**
  domain: NAV-AUM
  name: Cash vs NAV Consistency
  type: DATA_QUALITY
  description: Ensure total cash (per cash reconciliation) is included in NAV calculation. E.g., compare sum of all internal cash balances to difference between AUM and sum of non-cash assets.
  inputs:

  - total_cash_internal, total_cash_administrator (if available)
  - aum_internal, sum(position_values)
    logic_pseudocode_sql: | cash_from_nav = aum_internal - sum(position_values) MATCH if ABS(cash_from_nav - total_cash_internal) <= small_tol
    suggested_tolerances: amount_abs: 1.0 amount_pct: 0.1%
    date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Checks that our book-keeping is internally consistent.
    severity: level: HIGH
    impact_if_broken: Unnoticed cash errors would mean NAV includes/excludes cash incorrectly.
    example_breaks:
  - "Example: Sum positions $1,000, cash $10; AUM should $1,010. If AUM is $1,005, $5 missing means cash or valuation error."
    example_fix:
  - "Locate missing $5 (maybe a receivable or mis-posted fee) and include it to reconcile AUM."

- id: **NAV_004**
  domain: NAV-AUM
  name: P&L Consistency Check
  type: TOLERANCE
  description: Cross-check that the day's P&L (from trade/cash and market moves) is consistent with the NAV change. If NAV moved by X, sum of P&L on positions plus net cash flows should equal that (within tolerance).
  inputs:
    - prior_nav, current_nav, net_cash_flow, price_change_gain
      logic_pseudocode_sql: | nav_change = current_nav - prior_nav - net_cash_flow MATCH if ABS(nav_change - price_change_gain) <= tolerance_amount
      suggested_tolerances: amount_abs: 5.0 amount_pct: 0.05%
      date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Ensures no mystery P&L.
      severity: level: HIGH
      impact_if_broken: P&L errors or missing booking.
      example_breaks:
    - "Example: NAV up $100, but only $90 of trade gains + $5 cash flow explains $95; $5 unexplained."
      example_fix:
    - "Find $5 difference (could be forgot coupon or rounding), adjust records."

- id: **NAV_005**
  domain: NAV-AUM
  name: Foreign Security Valuation Check
  type: TOLERANCE
  description: If NAV includes securities in other currencies or markets, ensure conversion rates used match reference (custodian or standard source). Flag if discrepancy in local vs base valuation.
  inputs:
    - position_value_local, local_to_base_rate
    - custodian_local_value
      logic_pseudocode_sql: | base_val = local_value * FX(local->base) MATCH if ABS(base_val - custodian_local_value_converted) <= tol
      suggested_tolerances: amount_abs: 1.0 amount_pct: 0.1%
      date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Similar to position FX rules but looking at NAV impact level.
      severity: level: MEDIUM
      impact_if_broken: Minor, but multiple small errors accumulate NAV drift.
      example_breaks:
    - "Example: 1000 EUR position, rate 1.1000 gives $1100, custodian uses 1.1010 ($1101); $1 gap flagged."
      example_fix:
    - "Align FX source (maybe use custodian FX)."

- id: **NAV_006**
  domain: NAV-AUM
  name: Fee Accruals in NAV
  type: DATA_QUALITY

description: Verify that recurring fees (management fee accruals) have been included in NAV. For example, a 1.5% annual fee accrues daily. If NAV drop doesn't reflect expected fee, flag it.
inputs:

- aum_previous, aum_current, known_fee_rate, days
  logic_pseudocode_sql: | expected_fee = aum_previous * (annual_fee_rate/365 * days)
  MATCH if ABS((aum_previous - aum_current) - expected_fee) <= tolerance
  suggested_tolerances: amount_abs: 1.0 amount_pct: 0.05%
  date_offset_days: 0
  deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Simple accrual calculation.
  severity: level: HIGH
  impact_if_broken: Missing fee accrual leads to NAV understatement (for asset manager)!
  example_breaks:
- "Example: AUM $100M, 1% annual fee, 1 day accrues ~$2740. If NAV only reflects $2700 drop, $40 missing fee."
  example_fix:
- "Book the missing $40 as fee income to correct NAV; adjust accrual policy if needed."

- id: **NAV_007**
  domain: NAV-AUM
  name: Uninvested Cash Factor
  type: DATA_QUALITY
  description: Check if cash portions are too large/small. For example, if portfolio is supposed to be 100% invested but shows 20% cash, flag for review (possible orphan cash).
  inputs:

  - total_cash, total_AUM, expected_cash_pct (configurable)
    logic_pseudocode_sql: | cash_pct = total_cash / total_AUM * 100 IF cash_pct > expected_cash_pct_max THEN FLAG
    suggested_tolerances: amount_abs: 0 amount_pct: 0.5%
    date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Policies differ, but eg. equity funds usually <5% cash.
    severity: level: LOW
    impact_if_broken: Might be intentional (waiting funds), but very high cash suggests missing invests.
    example_breaks:
  - "Example: Equity fund NAV shows 15% cash (over threshold 5%), indicates new cash not invested."
    example_fix:
  - "Inform portfolio manager to invest or rebalance. If it's for a redemptions or distribution, mark as expected."

2.5 Fees & Charges Reconciliation Rules

- id: **FEES_001**
  domain: Fees
  name: Broker Commission Match
  type: TOLERANCE
  description: Verify that commissions recorded internally match the broker invoices or expected rates. If broker filed a commission of $X for a trade, the internal record should be $X. Small

rounding differences (a few cents) can be tolerated.
inputs:

- trade.commission
- cash.cash_amount labeled as commission or broker_invoice.commission
  logic_pseudocode_sql: | MATCH if ABS(trade.commission - cash.commission_amount) <= tol_abs
  suggested_tolerances: amount_abs: 0.01 amount_pct: 0.1% date_offset_days: 0
  deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Commission often appears as separate cash line or embedded.
  severity: level: MEDIUM
  impact_if_broken: Incorrect cost basis and P&L for trades.
  example_breaks:
- "Example: Broker invoice says $50 commission, internal blotter had $49.99; 1 cent diff (tolerable). $50.50 vs $49 (1.5% diff) breaks."
  example_fix:
- "Adjust internal commission to exactly match broker or allocate differences to fees expense."

- id: **FEES_002**
  domain: Fees
  name: Tax/Stamp Duty Validation (India-specific)
  type: DATA_QUALITY
  description: In Indian equity trades, confirm that Securities Transaction Tax (STT) and stamp duty are charged correctly. For example, STT on buys (0.1% of trade value) and on sales (0.1% sell-side).
  inputs:
  - trade.broker_code, trade.net_amount
  - cash.tax_amount lines (STT, stamp)
    logic_pseudocode_sql: | IF trade.market = 'India' AND trade.asset = 'Equity' THEN expected_STT = ABS(trade.net_amount) * 0.001 ASSERT ABS(cash.stt - expected_STT) < tol
    suggested_tolerances: amount_abs: 0.01 amount_pct: 0.01% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Requires region-specific logic. Mark as INDIA.
    severity: level: HIGH
    impact_if_broken: Regulatory violation if STT is missing/incorrect.
    example_breaks:
  - "Example: 1000 INR equity buy should have 1 INR STT; if none is recorded, flag. If 2 INR (double), also flag."
    example_fix:
  - "Insert or correct the STT charge. If calculation is tricky, tag for compliance review."

- id: **FEES_003**
  domain: Fees
  name: Management Fee Accrual Check
  type: DATA_QUALITY
  description: Verify that the periodic management fee has been accrued correctly. E.g., if 1% p.a.

monthly, the accrual should equal (AUM×1%/12).
inputs:

- fund.AUM, management_fee_rate, fee_accrued
  logic_pseudocode_sql: | expected_fee = AUM * fee_rate * (days/365) MATCH if ABS(fee_accrued - expected_fee) <= tol
  suggested_tolerances: amount_abs: 1.0 amount_pct: 0.01% date_offset_days: 0
  deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Similar to NAV fee rule; ensures fee recorded.
  severity: level: HIGH
  impact_if_broken: Undercharging fees hurts revenue.
  example_breaks:
- "Example: $10M AUM, 12% annual fee, 30 days -> expected $98,630; if only $98,000 accrued, short by $630."
  example_fix:
- "Manually accrue difference; check config for daily accrual vs monthly lumpsum."

- id: **FEES_004**
  domain: Fees
  name: Custody Fee Matching
  type: TOLERANCE
  description: Confirm that custody/custodian fees (often charged monthly/quarterly) match internal projections or invoice.
  inputs:

  - custodian_invoice.amount, internal.cost_centers.custody_fee
    logic_pseudocode_sql: | MATCH if ABS(invoice_fee - internal_fee) <= tol_abs
    suggested_tolerances: amount_abs: 0.5 amount_pct: 0.1% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Sometimes invoice covers multiple funds; compare per agreement.
    severity: level: MEDIUM
    impact_if_broken: Overpayment or missing expense; material if fees high.
    example_breaks:
  - "Example: Custodian billed $10,000, but internal was $9,800; $200 (2%) short."
    example_fix:
  - "Investigate invoice details, allocate correctly, adjust internal record."

- id: **FEES_005**
  domain: Fees
  name: Tax Withholding Verification (Corporate Dividends)
  type: TOLERANCE
  description: For foreign dividends, check that the tax withheld (if any) matches expected treaty rate. E.g., 10% of gross dividend.
  inputs:

  - corp_action.cash_amount (gross dividend), cash.tax_withheld
    logic_pseudocode_sql: | expected_tax = gross_dividend * withholding_rate MATCH if ABS(cash.tax_withheld - expected_tax) <= tol_abs
    suggested_tolerances: amount_abs: 0.01 amount_pct: 0.1% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Requires knowledge of country’s tax treaties; mark as GEOGRAPHY-SPECIFIC.
    severity: level: MEDIUM

impact_if_broken: Underwithholding violates tax law; overwithholding loses cash.
example_breaks:

- "Example: $100 dividend from US stock, expected $15 tax withheld; only $10 seen (underwithheld)."
  example_fix:
- "Adjust expected tax based on treaty or contact custodian for correction."

2.6 Corporate Actions Rules

- id: **CA_SPLIT_001**
  domain: Corporate Actions
  name: Stock Split Quantity Update
  type: DETERMINISTIC
  description: On a stock split (or reverse split), update the internal position quantity by the split ratio. E.g., a 2-for-1 split doubles quantity. This rule checks that the post-split quantity matches expectation.
  inputs:

  - corp_action.type = 'SPLIT', corp_action.ratio
  - position.quantity (pre- and post-ex-date)
    logic_pseudocode_sql: | IF today = corp_action.ex_date THEN expected_qty = old_qty * ratio MATCH if position.quantity = expected_qty suggested_tolerances: amount_abs: 0 amount_pct: 0% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Straight ratio multiply.
    severity: level: CRITICAL
    impact_if_broken: Wrong share count -> NAV and tax basis error.
    example_breaks:
  - "Example: 2-for-1 split, old_qty=100, expected new_qty=200. If internal still 100, break."
    example_fix:
  - "Double the quantity to 200 and halve the price/avg_cost. This clears the break."

- id: **CA_SPLIT_002**
  domain: Corporate Actions
  name: Cost Basis Adjustment (Split)
  type: DETERMINISTIC
  description: After a split, the average cost per share must adjust inversely (e.g. halved for 2-for-1). Check that cost basis updates accordingly.
  inputs:

  - pre_split_avg_cost, split_ratio
  - post_split_avg_cost
    logic_pseudocode_sql: | expected_cost = pre_split_avg_cost / ratio MATCH if post_split_avg_cost = expected_cost suggested_tolerances: amount_abs: 0.0001 amount_pct: 0.01% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Ensure no P&L effect.
    severity: level: HIGH
    impact_if_broken: If cost basis is wrong, realized gains will be incorrect.
    example_breaks:
  - "Example: Pre-split avg $10, after 2:1 split should be $5; if still $10, basis doubled wrongly."
    example_fix:
  - "Adjust cost to $5 (or $50 to $25 total invested)."

- id: **CA_BONUS_001**
  domain: Corporate Actions
  name: Bonus Issue Quantity Update
  type: DETERMINISTIC
  description: For a bonus share issue, additional shares are credited. Calculate expected new quantity = old_qty + (old_qty × bonus_ratio). Check match.
  inputs:
    - corp_action.type='BONUS', corp_action.ratio (e.g., 1 bonus per 4 held = 0.25)
    - position.quantity pre- and post-ex-date
      logic_pseudocode_sql: | IF today = corp_action.ex_date THEN expected_qty = old_qty * (1 + bonus_ratio) MATCH if position.quantity = expected_qty suggested_tolerances:
      amount_abs: 0 amount_pct: 0% date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Simple.
      severity: level: HIGH
      impact_if_broken: Wrong holdings count, NAV off.
      example_breaks:
    - "Example: 20% bonus (1-for-5): old 100, expected 120. If internal shows 100, missing 20 shares."
      example_fix:
    - "Add 20 shares to position, adjusting cost basis proportionally (reduce cost per share by 100/120)."

- id: **CA_DIV_001**
  domain: Corporate Actions
  name: Cash Dividend Receipt Check
  type: DETERMINISTIC
  description: On ex-dividend date, ensure that cash dividend (corp_action.cash_amount × quantity) appears in cash reconciliation.
  inputs:
    - corp_action.cash_amount (per share), position.quantity on record date
    - cash.amount of dividend payment
      logic_pseudocode_sql: | expected_div = corp_action.cash_amount * position.quantity MATCH if cash.amount (credit) = expected_div
      suggested_tolerances: amount_abs: 0.5 amount_pct: 0.1% date_offset_days: 1
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Allow 1-day lag (pay-date lag).
      severity: level: CRITICAL
      impact_if_broken: Missing dividend causes NAV understatement.
      example_breaks:
    - "Example: $1 dividend × 100 shares = $100. If only $90 credited, $10 short."
      example_fix:
    - "Check withholding taxes or dividend ratios; post missing $10 to cash if earned."

- id: **CA_RIGHTS_001**
  domain: Corporate Actions
  name: Rights Issue Entitlement
  type: DETERMINISTIC
  description: For a rights issue (right to buy additional shares), check that expected rights shares

are recorded if subscribed, or flagged if not exercised.
inputs:

- corp_action.type='RIGHTS', corp_action.ratio (rights per old share), corp_action.price
- position.quantity (to determine entitlement)
  logic_pseudocode_sql: | expected_rights = position.quantity * rights_ratio MATCH if position.rights_allocated = expected_rights or rights_declined flag
  suggested_tolerances: amount_abs: 0 amount_pct: 0% date_offset_days: 0
  deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Usually manual exercise; track entitlement.
  severity: level: MEDIUM
  impact_if_broken: If exercised rights not applied, ownership is wrong.
  example_breaks:
- "Example: 1-for-10 rights on 100 shares → 10 rights. If internal record has 0 or 5, break."
  example_fix:
- "Record exercised rights or log unexercised; adjust positions after settlement of rights."

- id: **CA_MERGER_001**
  domain: Corporate Actions
  name: Merger Stock Conversion
  type: EDGE_CASE
  description: In a simple merger/acquisition where shares convert (e.g., 3 old → 1 new), update holdings accordingly. (Complex M&A may not be fully handled.)
  inputs:

  - corp_action.type='MERGER', corp_action.ratio, new_symbol, old_symbol
  - positions pre- and post-date for old/new symbols
    logic_pseudocode_sql: | IF today = corp_action.effective_date THEN expected_new_qty = old_qty * (1/ratio) MATCH if position_new.quantity = expected_new_qty
    suggested_tolerances: amount_abs: 0 amount_pct: 0% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Simple conversions only. If minor cash consideration, separate rule.
    severity: level: HIGH
    impact_if_broken: Wrong holdings count, major P&L error.
    example_breaks:
  - "Example: Takeover of ABC by XYZ at 3:1. Old 300 shares should convert to 100 XYZ. If internal still shows 300 ABC, break."
    example_fix:
  - "Replace 300 ABC with 100 XYZ, adjust basis accordingly."

- id: **CA_PARAM_001**
  domain: Corporate Actions
  name: Ex-Date/Cash Date Tolerance
  type: TOLERANCE
  description: Sometimes the custodian posts positions the day after the official ex-date. Allow a 1-day offset when comparing pre- vs post-event positions.
  inputs:

  - corp_action.ex_date, corp_action.pay_date
  - position dates
    logic_pseudocode_sql: | MATCH positions on date = ex_date or (ex_date + 1 day)

suggested_tolerances: amount_abs: 0 amount_pct: 0% date_offset_days: 1
deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Reduce false break on weekends/holidays.
severity: level: LOW
impact_if_broken: False breaks if action posted next day.
example_breaks:

- "Example: Split ex-date 12/01, but internal system applied on 12/02; strict date match flagged break. With offset=1, accepted."
  example_fix:
- "Configure corp action processing to consider next day as effective if missing on ex-date."

2.7 FX & Multi-Currency Rules

- id: **FX_001**
  domain: FX
  name: FX Rate Tolerance
  type: TOLERANCE
  description: Check that the FX rate used to convert foreign transactions matches the source rate (e.g. custodian or central bank). Allow a small percentage tolerance for rounding.
  inputs:
  - recorded_fx_rate (used in trade or NAV)
  - official_fx_rate (e.g. ECB or custodian's rate)
    logic_pseudocode_sql: | diff_pct = ABS(recorded_fx_rate - official_fx_rate)/ official_fx_rate*100 MATCH if diff_pct <= 0.1%
    suggested_tolerances: amount_abs: 0 amount_pct: 0.1%
    date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: FX feeds may have minor differences.
    severity: level: MEDIUM
    impact_if_broken: Large rate differences cause NAV/Cash misstatement.
    example_breaks:
  - "Example: Recorded USD/EUR 0.9500, official 0.9508 (0.08% diff) OK; 0.9400 (1.05% diff) flagged."
    example_fix:
  - "Use a consistent rate (e.g. custodian's) or widen tolerance for specific currencies."

- id: **FX_002**
  domain: FX
  name: Base vs Local Currency Summation
  type: TOLERANCE
  description: For each account, sum all cash/positions converted to base currency and compare to reported base-currency totals.
  inputs:
  - list of cash/position amounts with currencies
  - FX table
    logic_pseudocode_sql: | total_base = SUM(amount_i * FX(currency_i->base)) MATCH if ABS(total_base - reported_base_total) <= tolerance
    suggested_tolerances: amount_abs: 5.0 amount_pct: 0.1%
    date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Ensures whole-account

consistency.
severity: level: HIGH
impact_if_broken: Indicates a missing FX entry or double count.
example_breaks:
- "Example: Total foreign cash $100k, base currencyUSD total reported $110k, FX sum $109.5k – $500 gap."
example_fix:
- "Check for an FX rounding difference or forgotten currency translation."

- id: **FX_003**
domain: FX
name: Unlikely FX Pair Check
type: DATA_QUALITY
description: Validate currency pair quotes. If an FX rate is outside a reasonable band (e.g. 20% away from prior day), flag it as possibly incorrect.
inputs:

  - today's FX_rate, prior day's FX_rate
  logic_pseudocode_sql: | IF ABS(FX_rate_today - FX_rate_yesterday)/FX_rate_yesterday > 20% THEN FLAG
  suggested_tolerances: amount_abs: 0 amount_pct: 20%
  date_offset_days: 0
  deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Catches bad feed.
  severity: level: LOW
  impact_if_broken: Rare spikes could break NAV.
  example_breaks:
  - "Example: EUR/USD leaps from 1.10 to 2.00 in one day (error) – flagged."
  example_fix:
  - "Re-import correct rates."

- id: **FX_004**
domain: FX
name: Cross-Currency Trade Matching (Indirect)
type: TOLERANCE
description: When neither currency of the trade equals the base, use a cross rate. E.g., trade in JPY, base USD; convert JPY→USD via USD/JPY or via EUR intermediate.
inputs:

  - trade.amount in foreign currency
  - FX rates (foreign->base, possibly via intermediate)
  - cash.amount base
  logic_pseudocode_sql: | expected_base_amt = trade.amount * FX(foreign->base) MATCH if ABS(cash.amount - expected_base_amt) <= tol_abs
  suggested_tolerances: amount_abs: 0.5 amount_pct: 0.1% date_offset_days: 0
  deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Must source correct cross FX.
  severity: level: MEDIUM
  impact_if_broken: Provides correct base values for reconciliation.
  example_breaks:
  - "Example: Buy 100,000 JPY (EUR base). Using JPY/EUR gives €700; if GBP/USD used by mistake, doesn’t match."
  example_fix:
  - "Use correct FX table (JPY→EUR rate) to convert."

- id: **FX_005**
  domain: FX
  name: Cumulative FX P&L Check
  type: TOLERANCE
  description: For positions held in FX (like FX forward or continuous exposure), verify that realized FX gains/losses plus mark-to-market equals the total P&L in base currency.
  inputs:

    - realized_fx_gain_loss, unrealized_fx_value_change
    - total_fx_pnl_reported
      logic_pseudocode_sql: | expected_pnl = realized + unrealized MATCH if ABS(expected_pnl - total_fx_pnl_reported) <= tol_abs
      suggested_tolerances: amount_abs: 1.0 amount_pct: 0.1% date_offset_days: 0
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Sanity check on FX derivates.
      severity: level: MEDIUM
      impact_if_broken: Could signal missed booking of an FX trade.
      example_breaks:
    - "Example: FX forward realized $100, unrealized $900, total $1000, but reported only $950 – missing $50."
      example_fix:
    - "Post missing $50 gain or find erroneous entry."

2.8 Suspense & Exception Management Rules

- id: **SUSP_001**
  domain: Suspense
  name: Unmatched Trade Aging
  type: EDGE_CASE
  description: Identify trades with no cash match after N days (e.g., 3 days past settlement) – these become aged breaks requiring manual investigation.
  inputs:

    - trade.settlement_date, current_date, match_status
      logic_pseudocode_sql: | IF today > trade.settlement_date + N AND trade.match_status = UNMATCHED THEN FLAG as aged break
      suggested_tolerances: amount_abs: 0 amount_pct: 0% date_offset_days: N (e.g., 3)
      deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Ensures timely resolution.
      severity: level: HIGH
      impact_if_broken: Aging breaks indicate unresolved operational risk.
      example_breaks:
    - "Example: A trade from Dec 1 (settle Dec 3) still unmatched by Dec 6 -> flagged."
      example_fix:
    - "Ops to investigate why cash is missing (e.g. cheque delay, settlement fail). Escalate if needed."

- id: **SUSP_002**
  domain: Suspense
  name: Unmatched Cash Aging
  type: EDGE_CASE

description: Similar to above, but for cash entries unallocated to any trade/position after M days.
inputs:

- cash.booking_date, match_status
  logic_pseudocode_sql: | IF today > cash.booking_date + M AND cash.match_status = UNMATCHED THEN FLAG
  suggested_tolerances: amount_abs: 0 amount_pct: 0% date_offset_days: M (e.g., 7)
  deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Stale cash (>7 days) is a red flag.
  severity: level: MEDIUM
  impact_if_broken: Idle cash can distort balances.
  example_breaks:
- "Example: $100 deposit dated one week ago still in suspense -> raised for manual review."
  example_fix:
- "If truly orphan, route to unidentified cash clearing."

- id: **SUSP_003**
  domain: Suspense
  name: Break Category Tagging (Initial)
  type: DATA_QUALITY
  description: Automatically tag breaks by suspected cause: DATA (invalid input), TIMING (settlement lag), ECONOMIC (amount diff), CORP_ACTION, COUNTERPARTY (unknown party), or OTHER.
  inputs:

  - characteristics of break (e.g. missing fields, date diff, exactly fees, corp_action context)
    logic_pseudocode_sql: | IF missing key fields THEN tag=DATA ELSE IF date_diff matches known lag THEN tag=TIMING ELSE IF amount_diff <= small_tol THEN tag=ECONOMIC ELSE IF corp_action present THEN tag=CORP_ACTION ELSE tag=OTHER
    suggested_tolerances: amount_abs: 5.0 amount_pct: 0.05% date_offset_days: 2
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Initial tagging helps routing. AI may refine reason later.
    severity: level: LOW
    impact_if_broken: Tagging doesn’t affect data, but ensures ops know why break occurred.
    example_breaks:
  - "Example: Missing ISIN → tag=DATA. 1-day later cash → tag=TIMING."
    example_fix:
  - "Review cause by tag; e.g., data-quality issues go to data team, otherwise ops solves."

- id: **SUSP_004**
  domain: Suspense
  name: Auto-Close Minor Rounding
  type: TOLERANCE
  description: Automatically close breaks where the difference is below a de minimis threshold (e.g. <$0.10 or 0.01%). Avoid ops overhead for trivial cents.
  inputs:

  - break.amount_diff
    logic_pseudocode_sql: | IF ABS(amount_diff) <= 0.10 THEN auto_resolve = TRUE
    suggested_tolerances: amount_abs: 0.10 amount_pct: 0.01% date_offset_days: 0
    deterministic_or_ai: should_live_in: "RULE_ENGINE" notes: Tier-2 tolerance rule to reduce

false break load.
severity: level: LOW
impact_if_broken: Negligible; omitting could clutter ops queue.
example_breaks:
- "Example: $1000 trade vs $1000.08 cash; 0.008% diff; auto-close allowed."
example_fix:
- "System marks as closed with reason “rounding”."

- id: **SUSP_005**
domain: Suspense
name: Bulk Break Pattern Detection (AI)
type: EDGE_CASE
description: Use AI to detect patterns in recurring breaks and suggest automated fixes (e.g. a broker who always mis-tags a fee line).
inputs:

- history of resolved breaks, manual fixes
logic_pseudocode_sql: | N/A (AI model analyzes break log and fix steps)
suggested_tolerances: amount_abs: N/A amount_pct: N/A date_offset_days: N/A
deterministic_or_ai: should_live_in: "AI_LAYER" notes: Tier-3: periodic AI analysis of resolution logs to update rules (Section 6.3).
severity: level: MEDIUM
impact_if_broken: Without it, repetitive manual work.
example_breaks:
- "Example: Every Friday, broker X has a $12 fee that ops manually maps. AI can learn and automate mapping rule."
example_fix:
- "Not a day-to-day fix; an AI ‘learning event’ updates the rulebook."

SECTION 3 — CORPORATE ACTIONS & SPECIAL DAYS

3.1 Corporate actions to handle: We include the common types:
- **Stock Splits (and Reverse Splits):** Ratio-based change in share count (e.g., 2-for-1 split doubles shares; 1-for-3 reverse splits reduce shares).
- **Bonus (Rights) Issues:** Issuance of free shares to existing holders (e.g., 1 new share per 5 held). Note: in India, bonus and rights are often used interchangeably; rights typically involve a purchase.
- **Cash Dividends:** Payment per share on ex-date/paid-date.
- **Rights Issues (simple):** Offer to buy additional shares at discount (not deeply modeled here, assume either accepted or lapsed, see Section 2 CA_RIGHTS).
- **Mergers/Demerger:** High-level handling: share conversions (e.g., 3 old → 1 new). For demergers (spin-offs), assume a single new entity. We do not cover full complexities of multiple spinoffs.

3.2 Impact of each action:

- **Stock Split (and Reverse):**
- *Quantity:* Increase or decrease by ratio.
- *Price/Cost Basis:* Price is divided (or multiplied) by same ratio; total cost stays constant (so average cost adjusts).

- *Holdings Table:* Quantity ← Quantity × ratio; market_price ← market_price / ratio (approx).
- *Trade/Cash Reconciliation:* Trades around split date may have ratio-adjusted volumes; our split rules ensure no false breaks. E.g. trades on T and T-1 should consider split if settlement crosses ex-date.
- *PNL/AUM:* No economic change aside from rounding, so NAV per unit halved (for 2-for-1) and overall AUM stays equal. Check rules prevent false NAV breaks (e.g., NAV per share*2 = old NAV if ratio 2).

- *Ex-date Rules:* On ex-date, ensure matching uses *post*-split quantities. Corporate action rules (e.g. CA_SPLIT_001/002) catch misalignments.

- **Bonus Issue:**

- *Quantity:* Increase by bonus ratio (similar to split). No cash outlay to shareholders.
- *Cost Basis:* Total cost is spread over more shares; price per share adjusts down accordingly.
- *Holdings:* Like split, quantity update.
- *PNL/AUM:* AUM unchanged, NAV per share falls.

- *Recon Rules:* E.g. after a 20% bonus, internal should have 20% more shares. Edge-case: if bonus goes to a subset (like only new investors), handle if needed (beyond scope).

- **Cash Dividend:**

- *Quantity:* Unchanged.
- *Cash:* A cash credit appears on pay-date. Entry should reconcile to corp_action.cash_amount×quantity.
- *NAV:* NAV drops by total dividend (prepaid). Ensure cash flows align.
- *Ex-date/Record-date Rules:* On ex-date, the holding count locks in; payout on pay-date. We allow matching cash up to a few days after ex-date (see NAV_003 and CA_DIV).

- *False Breaks:* Avoid flagging NAV differences as errors if the expected dividend was not yet paid. The rule CA_DIV_001 accounts for this by matching expected vs actual.

- **Rights Issue:**

- *Quantity:* If rights exercised, new shares added at subscription price.
- *Cash:* Paying price for rights uses cash; if not exercised, no cash.
- *Holdings Update:* If exercised, holdings increase; if lapsed, no change. Complex to automate.
- *NAV/AUM:* Adjust for new capital; small net cash change (exercise price × rights).

- *Ex-date:* Record-date yields entitlement. Our CA_RIGHTS_001 tags whether rights were taken up. If some are unexercised, this appears as nothing, and is not a break (just fewer shares).

- **Merger/Acquisition:**

- *Quantity:* Convert old shares to new ratio.
- *Cash:* Often a cash portion (not handled here) or all-share deal; we assume simple swap.
- *Holdings:* Delete old ticker, create new ticker with converted qty.
- *NAV:* No P&L effect (except spread). If cash part exists, our cash rules (Trade-Cash) catch that.
- *Rules:* CA_MERGER_001 handles basic share conversion. Record date adjustments similar to splits.

3.3 Example Rule IDs for Corporate Actions:
We have exemplified many above. Additional rule specs could include:

- **CA_DIV_002:** Validate dividend currency. If fund base is USD but dividend declared in JPY, ensure FX conversion is applied. (AI if multiple currency feeds needed).
- **CA_SPLIT_003:** Multi-step splits. If splits announced in two tiers, verify sequential application.
- **CA_RIGHTS_002:** Detect unexercised rights expiration (if rights are for cash and not applied, no break but note for reporting).

(Any highly specialized corp action rules should be marked “NEEDS VALIDATION” until local compliance confirms details.)

SECTION 4 — FX & MULTI-CURRENCY LOGIC

4.1 Multi-currency reconciliation: When trades are in one currency and cash in another, or positions held in foreign securities, Aureon must convert consistently. For each trade with currency ≠ base_currency, use the correct FX rate (e.g., custodian’s official end-of-day rate) to convert cash flows into base currency [1]. Cash from FX trades should reconcile via the same rate. Common FX breaks are due to wrong source (bank rate vs market), stale date, or rounding. Similarly, holdings in foreign stocks should be valued in base currency for NAV; differences vs local custodian values should be explained. We assume access to daily FX rates from a provider; rules will use these.

4.2 FX rules: Examples below (full list in Section 2).
- *FX tolerance:* FX_001 (above) checks rate diff within 0.1%. If exceeded, it’s probably a wrong rate choice.
- *Rounding:* Many FX calculations need cent rounding. We allow a few cents per large trade (see FX_001).
- *Cross-currency PnL:* FX_005 ensures realized + unrealized equal total.
- *FX platform:* Aureon should integrate an FX feed (Reuters, Bloomberg, or custodian) daily. For interim, possibly two-point interpolation (for midnight crosses). This is an implementation detail.
- *Multi-currency accounts:* If a fund holds multiple currencies, reconcile each currency’s cash separately, then aggregate to base. Cross-currency netting (e.g., EUR sale cash offsetting USD purchase) should be allowed only if a legitimate FX exchange was made (often disallowed; better to handle via separate FX trade and cash).

Example rule spec already given above; we would add a few more if needed (e.g., FX deal-specific tolerances), but many FX issues are covered in Sections 2 and 5.

SECTION 5 — EXCEPTION MANAGEMENT & WORKFLOWS

5.1 Exception lifecycle: When a break is detected by the rule engine, it enters the exception workflow: - **Auto-resolved:** Some breaks are auto-closed if they meet tolerance (e.g. rounding) or deterministic fixes (e.g. matched partial fills). These should not go to Ops queue.
- **Human review:** Unresolved breaks are assigned to an Operations analyst, who investigates. They may use system suggestions (see Section 6) or audit trails.
- If the break is severe (e.g. material NAV impact, regulatory trade mismatch), it may escalate to PM or Compliance for sign-off after Ops investigation.
- **Aging:** We track age: e.g., break is flagged as "1-day old, 3-day old" etc. Policy: if unresolved >3 business days, send alert; >7 days, escalate. (Exact thresholds configurable per fund).

- The workflow may include comments, assignment fields, and category tags (see below). All actions logged (for audit and learning).

5.2 Break categories: To streamline processing, categorize each break: - **DATA:** Caused by missing/invalid data (e.g. missing ISIN, incorrect dates).
- **TIMING:** Caused by settlement lag or expected delays (e.g. T+1 effect).
- **ECONOMIC:** Minor differences (e.g. small rounding or negligible amount diff).
- **CORPORATE_ACTION:** Difference due to a known corporate event (split, dividend, etc.).
- **COUNTERPARTY:** Issues like missing counterparty (e.g. bank entry with unknown desc).
- **OTHER:** Catch-all for uncategorized.

These tags help prioritize: e.g. CORP_ACTION often low-impact (expected), whereas COUNTERPARTY (unknown trade) is serious.

5.3 Automation vs Manual:
- *Auto-closed by rules:* Rounding mismatches (see SUSP_004), exact internal loopbacks, trivial data fixes (e.g. trim whitespace) can be automated.
- *Human queue:* - CRITICAL breaks (NAV affecting, P&L impacting) always to human, even if matching candidate exists.
- Unmatched trades/cash beyond tolerance or missing fields.
- Any cross-system reconciliation (trade vs custodian) not auto-linked.
- *Alerts:* Anything CRITICAL or HIGH severity (see rule specs) triggers an immediate alert to ops management (e.g. email or dashboard notification). Example: If NAV_001 break >0.1%, or an unmatched large trade > $1M. Low-level breaks (like tiny rounding) just appear in exception inbox with low priority.

Breaks should be aged and flagged: e.g. a “Critical” break that remains open 1-day sends a reminder; 3-day escalation. Risky patterns (e.g. recurring same break) should be reported monthly.

SECTION 6 — HOW TO PARTITION LOGIC: RULE ENGINE vs AI vs SELF-HEALING

6.1 Which rules live where:
- **Deterministic Rule Engine:** All Tier-1 rules (exact matches, arithmetic identities) belong here. Examples: TRADE_CASH_001 (exact amounts), POS_CUST_001 (exact quantity), NAV_001 (NAV equality within tolerance), CA_SPLIT_001, FX_001, etc. These are implemented directly in SQL/Python.
- **Ideal AI/LLM candidates:** Fuzzy or semantic tasks, such as matching free-text descriptions, multi-mapping of symbols, or interpreting unusual transactions. For example: unmapped broker narrative fields, messy PDF contract notes, aliasing of instruments, or recommending match among multiple candidates (e.g. AI could read 'XYZ Corp sale' note and match to trade by semantic similarity). Also periodic analysis of break resolutions (self-healing).
- **Hybrid:** Cases like partial netting (TRADE_CASH_008): rule engine can propose candidate groupings by brute force or SQL, but AI can choose the correct grouping. Another hybrid: Rule engine filters a small list of potential matches; LLM chooses best match or recommends fix (in JSON as current Aurion agent does). The spec calls these out in `deterministic_or_ai.notes` (see many above labeled HYBRID).

6.2 Rule Engine API (suggested functions): (Outline of 20 functions to be implemented in Python or SQL library.)
1. **match_amount(trade_amount, cash_amount, tol_abs, tol_pct):** Returns true if amounts equal within tolerances.

2. **match_date(trade_date, cash_date, max_offset_days):** True if dates match or are within allowed offset.
3. **match_currency(trade_currency, cash_currency):** True if currencies match or convertible.
4. **match_quantity(position_qty, custodian_qty, tol_qty):** Check share quantities (maybe exact or with small tolerance for corporate actions).
5. **match_nav(nav_internal, nav_external, tol_pct):** Check NAV equality within tol.
6. **calculate_expected_cash(trade):** Return expected cash flow (with sign) for a trade.
7. **find_unmatched_items(internal_list, external_list):** Return items in internal not in external (for suspense).
8. **aggregate_by_key(records, key_fields):** Group records by keys (e.g. date+symbol). Useful for netting.
9. **match_commission(trade_comm, cash_fees, tol):** Check commission consistency.
10. **find_partial_matches(trade, cash_list):** Return list of cash entries that could partially settle trade (for AI review).
11. **match_positions(position, custodian_holding):** Compare quantity/value and return match score.
12. **apply_split(position, split_ratio):** Adjusts internal position for a split.
13. **calculate_nav(positions, cashes, fees):** Recompute NAV. (For cross-check.)
14. **compute_fx_value(amount, rate):** Return converted amount.
15. **find_aliases(symbol, external_list):** Suggest matching symbol (could call LLM or alias table).
16. **parse_description(text):** Extract structured info from free text using NER/LLM (e.g., identify trade IDs or fees).
17. **flag_break(break_type, severity):** Mark break with category and priority.
18. **update_holdings_for_ca(holding, corp_action):** Apply a corporate action to a holding.
19. **calc_expected_fee(aum, rate, period):** Compute management fee expected.
20. **suggest_fix(break_record):** (AI-layer) Given a break, suggest likely fix in JSON (update field, link trades, etc.).

6.3 Self-Healing:
Aureon will log every manual exception resolution as a **learning_event** (tie trade/cash IDs to human fix actions and commentary). Periodically (e.g. weekly), an AI batch job will analyze these events to improve the system:
- **Tolerances:** If many $0.50 breaks were auto-closed, AI might relax the auto-close threshold to 1.0. Or if multiple $10 undetected slippages, tighten rules.
- **Broker quirks:** If a broker always reports commission in a separate line, AI can create a rule to aggregate that.
- **Symbol mappings:** If an analyst repeatedly matches “Ticker AAAX” with “ISIN BBB”, the system can add an alias mapping rule.
- **Recurring pattern rules:** For example, if every Friday certain account always has a suspense $100 entry, AI might classify it as “sweep” and auto-clear.
- The outcome: AI proposes new rules or parameter adjustments, which can be reviewed (or automatically inserted) into a **RULE_MEMORY** table. Future runs incorporate these learned refinements. This is a Tier-3 feedback loop that gradually reduces manual exceptions. (Care must be taken to validate changes – possibly through a sandbox testing step.)

SECTION 7 — PRIORITIZED IMPLEMENTATION ROADMAP

We propose a phased rollout of the reconciliation engine, focusing first on high-impact rules and domains, then adding complexity.

Phase 1 (MVP, 4–6 weeks):
- **Domains:** Start with **Trade ↔ Cash** and **Positions ↔ Custodian** for equities (US/India). These yield the most daily breaks. Also minimal NAV sanity checks (NAV_001 with wide tolerance).
- **Rules:** Implement core deterministic rules: TRADE_CASH_001–005, POS_CUST_001–003, basic TOLERANCE rules (NAV_001 & NAV_002 with 0.1% tol), FX_001 for cross-currency. Also implement Tier-0 data checks (missing fields, invalid currency).
- **Assumptions:** Only cash and equity trades; ignore derivatives and corporate actions initially.
- **Metrics:** Aim to auto-resolve at least 50–70% of breaks (all clear cases), reducing manual breaks by half. Track: % of trade-cash matches automated, manual break count day-1 vs pre-launch. Example target: 70% auto-resolution of cash breaks, 80% of simple position matches.
- **Tools:** Build basic exception dashboard for analysts.
- **Success:** Few high-severity breaks (overrides must drop by X%). Time saved: if ops currently spend ~3 hours/day on errors, aim for 1.5h (50%) reduction.

Phase 2 (Next 2 months):
- **Add corporate actions (splits, dividends), FX rules, fees:** Implement CA_SPLIT_001/002, CA_DIV_001, CA_BONUS_001 etc. Add FX conversions (FX_002..). Expand fees: FEES_001–003.
- **Edge cases:** Partial settlement, multi-match (TRADE_CASH_007–008), autop-close rules, suspense aging logic.
- **Workflows:** Implement break tagging and basic queue (per Section 5). Build notifications for critical breaks.
- **Assumptions:** Add ability to handle T+2 flows, simple derivatives futures (direction only).
- **Metrics:** Auto-resolve rate > 80%. Exceptions sorted into categories for reporting. Operational time cut by another 20%.
- **Data Learning:** Start collecting learning_events. Possibly pilot an AI session for partial-match suggestions on a subset.
- **Outputs:** Dashboards showing break aging, categories. Document common fixes to refine rules.

Phase 3 (Long-term, 6–12 months):
- **Full multi-asset & multi-region:** Add mutual funds, bonds, options, futures. Regional tax rules (like STT, transaction taxes) configurable.
- **Advanced AI:** Deploy LLMs for complex matching: parsing unstructured docs, suggesting matches for ambiguous breaks. Implement self-healing: periodic LLM analysis of learning_events to auto-generate new rules (populate RULE_MEMORY).
- **Analysis:** Advanced analytics (break root-cause statistics, broker performance metrics).
- **Metrics:** Aim > 90% auto-resolution overall. Exception aging to near zero (all breaks resolved same day or auto-cleared).
- **Regulatory:** Prepare audit trail and documentation for compliance (e.g. SEC/T+1, EMIR).
- **Milestones:** Validate system with large-multi fund pilot (BlackRock-style scale: millions of transactions).

Each phase success should be measured in terms of: - **Break reduction:** % of cases auto-matched vs manual.
- **Ops efficiency:** time saved per funds team (log current vs after automation).
- **Accuracy:** No material NAV errors or regulatory incidents caused by the engine.

By this roadmap, Aureon launches a lean but useful reconciler quickly, then iteratively covers more cases, eventually offering tier-1 fund-scale automation with AI-enhanced flexibility.

Sources: Industry best practices for cash/position/NAV reconciliation [1] [2] [6] [4]; global settlement conventions [7]. (Specific rule logic is based on typical fund accounting operations.)

1 7 Cash Reconciliation: A Comprehensive Guide
https://www.limina.com/blog/cash-reconciliation-guide

2 Understand The Role of Position and Cash Reconciliation - Indus Valley Partners
https://www.ivp.in/resources/articles/understand-the-role-of-position-and-cash-reconciliation/

3 5 6 What is Securities Reconciliation?
https://www.solvexia.com/glossary/securities-reconciliation

4 What is NAV Reconciliation?
https://www.solvexia.com/glossary/nav-reconciliation